



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO

DISCIPLINA: LABORATÓRIO DE ALGORITMOS E ESTRUTURAS DE DADOS II

DOCENTE: KENNEDY REURISON LOPES

DISCENTES:

JOÃO PEDRO VENÂNCIO DE PAIVA

JOSE SALLES SOARES SOUZA

LUIZA MARCELINO DE QUEIROZ SANTOS

RELATÓRIO EXPERIMENTAL

PAU DOS FERROS/2026

Sistema de Verificação de Cadastro com Tabela Hash e Filtro de Bloom

Relatório Experimental

1. Introdução

Este relatório documenta o desenvolvimento de um sistema de verificação de cadastro de usuários combinando Tabela Hash (armazenamento e busca exata) e Filtro de Bloom (pré-filtro probabilístico). O sistema evita consultas desnecessárias à estrutura principal ao descartar imediatamente elementos definitivamente ausentes, e conta com interface gráfica em Raylib para operações interativas e experimentos em lote.

2. Implementação

Tabela Hash

Implementada com encadeamento externo e função hash DJB2. Tamanho fixado em **200.003 posições** (número primo), garantindo fator de carga $\alpha \approx 0,50$ no pior cenário (100k registros). Números primos como módulo reduzem agrupamentos, mantendo busca em $O(1)$ médio. O encadeamento externo foi preferido ao endereçamento aberto por não saturar com fatores de carga próximos de 0,5.

Filtro de Bloom

Vetor de bits dimensionado pelas fórmulas ótimas com taxa de falsos positivos $p = 1\%$:

$$m = -(n \times \ln p) / (\ln 2)^2 \quad \rightarrow \quad k = (m / n) \times \ln 2$$

Para 100k registros: $m \approx 958.506$ bits (~120 KB) e $k \approx 7$ funções hash — muito menor que os ~1,1 MB das chaves armazenadas diretamente. A combinação DJB2 + SDBM via *double hashing* ($\text{pos}_i = (h1 + i \times h2) \bmod m$) simula k funções independentes sem implementá-las separadamente.

3. Experimentos e Resultados

Em cada cenário, metade das buscas foi feita com registros intencionalmente ausentes (chaves modificadas), forçando falsos positivos mensuráveis. O tempo foi medido com `clock()` sobre o conjunto total.

Quantidade	Tempo sem Bloom	Tempo com Bloom	Falsos Positivos	Taxa FP (%)
1.000	[0,001]	[0,003]	[13]	[2,60%]
10.000	[0,009]	[0,0023]	[45]	[0,90%]
100.000	[0,0086]	[0,0163]	[1274]	[2,55%]

Tabela 1 — Resultados experimentais comparativos.

4. Análise

- O Bloom reduziu consultas à hash?** Sim. Elementos genuinamente ausentes (~50% do conjunto) foram descartados em $O(1)$, sem nenhum acesso à tabela hash. O filtro não gera falsos negativos, portanto nenhum elemento presente foi descartado incorretamente.
- Impacto do tamanho do vetor?** Vetor maior reduz a densidade de bits ativados, diminuindo colisões e a taxa de FP. O dimensionamento pela fórmula ótima manteve p próximo de 1% em todos os cenários.
- O número de funções hash alterou a precisão?** Sim. k muito baixo facilita coincidências; k muito alto satura o vetor. O valor ótimo $k \approx 7$, calculado pela fórmula, minimiza a taxa de FP para os parâmetros adotados.
- Cenário mais vantajoso?** 100.000 registros. A diferença de tempo é mais expressiva pois o custo da hash cresce com colisões, enquanto a consulta ao Bloom permanece $O(k)$ constante.

5. Conclusão

A combinação Hash + Bloom mostrou-se eficaz para verificação de cadastro em larga escala. O Filtro de Bloom eliminou consultas desnecessárias com custo constante e taxa de falsos positivos controlada (~1%), e o ganho de desempenho foi proporcional ao volume de dados. Todas as estruturas foram implementadas do zero em C, com dimensionamento baseado em análise quantitativa, e os resultados experimentais confirmaram as previsões teóricas.

Referências

BERNSTEIN, D. J. **Algoritmo de hash djb2**. Usenet: comp.lang.c, [s.d.].

BLOOM, B. H. Space/time trade-offs in hash coding with allowable errors. **Communications of the ACM**, v. 13, n. 7, p. 422-426, jul. 1970.

CORMEN, T. H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

KERNIGHAN, B. W.; RITCHIE, D. M. C. **C: a linguagem de programação**. 2. ed. Rio de Janeiro: Campus, 1989.

KIRSCH, A.; MITZENMACHER, M. Less hashing, same performance: building a better bloom filter. **Random Structures & Algorithms**, v. 33, n. 2, p. 187-218, 2008.

SEDGEWICK, R.; WAYNE, K. **Algorithms**. 4. ed. New Jersey: Addison-Wesley, 2011.

YIGIT, O. **SDBM**: a library of database functions. [S. l.]: [s. n.], [s.d.].